## 2. Blocks

- Block is a set of instruction defined in curly braces.
- Block does not have any name so you can't invoke the block explicitly.
- Block will be invoked by the JVM automatically.
- Block defined inside the class directly as a member are called as INITIALIZATION BLOCK.
- There are three type of block
  I. Instance Initialization Blocks
  II. Static Initialization Blocks
  III. Local block.

### I. Instance Initialization Block

- Block defined in the class without using static modifier are called as INSTANCE INITIALIZATION BLOCK (IIB).
- IIB will be invoked by the JVM automatically after initializing the **object**.

```
class hello{
 {
    System.out.println(" IIB ");
 }
}
```

- IIB will execute more than one time.
- Instance block will execute every time when object is created.
- Instance variable must be initialized within instance block only.
- Instance variable can't be accessed from static context directly but can be accessed with object reference.
- Class can contain only five class member; you can't place any other java Statement directly.
  Example:

```
class Hello {
  int a;          //valid
  a=10;           //Invalid
}
```

```
class Hello {
  int b=10;                  //valid
  System.out.println(b);     //Invalid
}
```

### II. Static Initialization Block

- Block define in the class using static modifier are called as static Block.
- SIB will be invoked by the JVM automatically after initializing the **class**.

```
class Hello {
static {
     System.out.println(" SIB ");
  }
}
```

- Static variable should be initialized within Static block only but there is a chance to initialized static variable within instance block also.

- Static block will be executed only once when class is loaded.
- Class will load only once when your JVM use the class members first time.
- Instance variable can't be accessed from static context directly but can be accessed with object reference.

**Programs:**

**OOPS25.java**

```
class OOPS25{
public static void main(String args[]){
int a;
a=10;
System.out.println(a);
  }
}
```

**OOPS26.java**

```
class OOPS26{
public static void main(String args[]){
Hello h=new Hello();
System.out.println(h.a);
 }
}
class Hello{
int a;
a=10;
}
```

**OOPS27.java**

```
class OOPS27{
public static void main(String args[]){
System.out.println(Hello.a);
  }
}
class Hello
{
static int a;
a=10;
}
```

**OOPS28.java**

```
class OOPS28{
public static void main(String args[]){
System.out.println("Main:"+Hello.a);
  }
}
class Hello{
static int a;
 {
  a=10;
  System.out.println("initialized :"+a);
  }
}
```

**OOPS29.java**

```
class OOPS29{
public static void main(String args[]){

Hello h=new Hello();
System.out.println("Main:"+h.a);
  }
}
class Hello{
int a;
{
a=10;
System.out.println("initialized :"+a);
        }
}
```

**OOPS30.java**

```
class OOPS30{
public static void main(String args[]){
System.out.println("Main:"+Hello.a);
  }
}
class Hello    {
static int a;
static{
    a=10;
    System.out.println("initialized :"+a);
   }
}
```

**OOPS31.java**

```
class OOPS31{
public static void main(String args[]){
System.out.println(Hello.a);
System.out.println(Hello.a);
   }
}
class Hello{
static int a=10;
{
   System.out.println("instance Block ");
}
static{
    System.out.println("Static Block ");
   }
}
```

**OOPS32.java**

```
class OOPS32{
public static void main(String args[]){

Hello h=null;
System.out.println("ref Created");
      }
}
class Hello{
static int a=10;
  {
   System.out.println("Instance Block ");
  }
static{
     System.out.println("Static Block ");
   }
}
```

**OOPS33.java**

```
class OOPS33{
public static void main(String args[]){

Hello h=null;
System.out.println("Ref Created");
h=new Hello();
   }
}
class Hello{
{
  System.out.println("Instance Block :");
}
static{
  System.out.println("Static Block :");
   }
}
```

**OOPS34.java**

```
class OOPS34{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Ref Created");
new Hello();
   }
}
class Hello{
{
    System.out.println("Instance Block ");
}
static{
   System.out.println("Static Block ");
    }
}
```

**OOPS35.java**

```
class OOPS35{
public static void main(String args[]){

System.out.println("Main :"+Hello.a);
}
}
class Hello   {
  static int a;
  {
   a=10;
   System.out.println("instance block :");
   }
}
```

**OOPS36.java**

```
class OOPS36{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main :"+Hello.a);
  }
}
class Hello{
static int a;
  {
  a=10;
  System.out.println("instance block :");
   }
}
```

**OOPS37.java**

```
class OOPS37{
public static void main(String args[]){
System.out.println("Main :"+Hello.a);
   }
}
class Hello     {
static int a;
int b;
static {
   a=10;
   b=20;
   System.out.println("Static block :");
  }
}
```

**OOPS38.java**

```
class OOPS38{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main :"+h.b+" \t "+h.a);
   }
}
class Hello{
static int a;
int b;
static {
  a=10;
   Hello h=new Hello();
   h.b=20;
  System.out.println("S B "+h.b+"\t"+ h.a);
   }
}
```

**OOPS39.java**

```
class OOPS39{
public static void main(String args[]){
System.out.println("Main ");
}
static  {
     System.out.println("Static block :");
   }
}
```

**OOPS40.java**

```
class OOPS40{
public static void main(String args[]){
System.out.println("Main :"+Hello.a);
    }
}
class Hello{
static int a;
static  {
   System.out.println("Static Block 1 ");
}
static  {
 System.out.println("Static Block 2 ");
   }
}
```

**OOPS41.java**

```
class OOPS41{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Hello");
   }
}
class Hello{
  {
    System.out.println("IIB block 1 ");
  }
  {
    System.out.println("IIB block 2");
  }
}
```

**OOPS42.java**

```
class OOPS42{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main :"+h.a);
  }
}
class Hello{
{
  System.out.println("IIB block 1 ");
   }
{
  System.out.println("IIB block 2 ");
   }
}
```

**OOPS43.java**

```
class OOPS43{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main :"+h.a);
  }
}
class Hello{
final int a=99;
}
```

**OOPS44.java**

```
class OOPS44{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main :"+h.a);
  }
}
class Hello{
final int a;
 {
   a=99;
  }
}
```

**OOPS45.java**

```
class OOPS45{
public static void main(String args[]){
System.out.println("Main :"+Hello.a);
}
}
class Hello{
      static final int a;
}
```

**OOPS46.java**

```
class OOPS46{
public static void main(String args[]){

System.out.println("Main :"+Hello.a);
}
}
class Hello{
          static final int a=99;
}
```

**OOPS47.java**

```
class OOPS47{
public static void main(String args[]){
System.out.println("Main : "+Hello.a);
  }
}
```

```
class Hello{
static final int a;
static{
   a=99;
  }
}
```

## Program output /error and details:-

| Prog No. | Output | Error and Details |
|---|---|---|
| OOPS25 | 10 | Here, "a" is a local variable so you can declare and initialize in two lines. |
| OOPS26 | Compilation error | [Error: <identifier> expected (a=10 ;)] Because, "a" is a Instance variable so you can not declare and initialize in two line. |
| OOPS27 | Compilation error | [Error: <identifier> expected (a=10 ;)] Because, "a" is a Static variable so you can not declare and initialize in two line. |
| OOPS28 | Main: 0 | Here, Object is not creating so IB will not executing. |
| OOPS29 | Initialized :10<br>Main:10 | Here, IIB will be invoked by the JVM automatically after initializing the **object** then Main method statement will execute. |

| Prog No. | Output | Error and Details |
|---|---|---|
| OOPS30 | Initialized :10<br>Main:10 | SIB will be invoked by the JVM automatically after initializing the **class**. |
| OOPS31 | Static Block<br>Instance Block<br>Instance Block | Static block will be executed only once when class is loaded. IIB will execute more than one time. Instance block will execute every time when object is created. |
| OOPS32 | Ref Created | Here, class will not load because Class will load only once when your JVM use the class members first time and you are not using any class member. |
| OOPS33 | Ref Created<br>Static Block<br>Instance Block | Here, class will load because JVM use the class members before that reference will created then object created so that SB and IB will be executed. |
| OOPS34 | Static Block<br>Instance Block<br>Ref Created<br>Instance Block | Instance block will executed two time because object is create two time. |
| OOPS35 | Main : 0 | Static variable should be initialized within Static block only but there is a chance to initialized static variable within instance block also. |
| OOPS36 | Instance block<br>Main : 0 | Static variable should be initialized within Static block only but there is a chance to initialized static variable within instance block also. Instance block will also executed due to object creation |
| OOPS37 | Compilation error | [Error: non-static variable b cannot be referenced from a static context] we cannot initialize instance variable in static Block. |
| OOPS38 | Static block :20<br>10<br>Main :0  10 | We can initialize instance variable in static Block when object created inside the block. |
| OOPS39 | Static block<br>Main | Here, 1st static block will executed then main method statement. |
| OOPS40 | Static block 1<br>Static block 2<br>Main: 0 | Here, 1st static block will executed then main method statement. |
| OOPS41 | IIB block 1<br>IIB block 2<br>Hello | Here, class will load 1st then object created so that instance block will be executed. |
| OOPS42 | Compilation error | [Error: variable a might not have been initialized]<br>Final Variable has to initialize. |
| OOPS43 | Main : 99 | Final Variable has to initialize. |
| OOPS44 | Main :99 | Final Variable has to initialize. |
| OOPS45 | Compilation error | [Error: cannot find symbol a]. Static Final Variable has to initialize. |
| OOPS46 | Main :99 | Static Final Variable has to initialize. |
| OOPS47 | Main :99 | Static Final Variable has to initialize in static block. |

## III. Local Block

- Block can be declared at two places
  - ⇒ Inside the class directly as a member
  - ⇒ Inside the members of the class
- Block defines inside the class members like methods, Constructors and blocks are called as Local Block.
- Local block will executed only when the enclosing class member is executed.

**Programs:**

**OOPS48.java**

```
class OOPS48{
public static void main(String args[])
{
System.out.println("main: "+Hello.a);
  }
}

class Hello{
static int a;
 {
  System.out.println("IB ");
  {
   System.out.println("LB in inside IB ");
  }
 }
static{
  System.out.println("SB ");
  {
   System.out.println("LB in inside SB");
  }
  }
}
```

**OOPS49.java**

```
class OOPS49{
public static void main(String args[])
{
new Hello();
System.out.println("main “);
  }
}

class Hello{
static int a;
 {
  System.out.println("IB");
  {
   System.out.println("LB in inside IB ");
  }
 }
static{
  System.out.println("SB: ");
  {
   System.out.println("LB in inside SB: ");
  }
  }
}
```

**OOPS50.java**

```
class OOPS50{
public static void main(String args[]){
System.out.println("Main() ");
{
  System.out.println("LB in Main () ");
  }
 }
}
```

**OOPS51.java**

```
class OOPS51{
public static void main(String args[]){

System.out.println("Main() ");
{
  System.out.println("LB1 in Main () ");
 }
 {
   System.out.println("LB2 in Main () ");
  }
 }
}
```

**OOPS52.java**

```
class OOPS52{
public static void main(String args[]){
 Hello h=new Hello();
System.out.println("Main() "+h.a);
  }
}
class Hello{
  {
   int a=10;
   String a="IND";
   System.out.println("I.B : "+a);
  }
}
```

**OOPS53.java**

```
class OOPS53{
public static void main(String args[]){

System.out.println("Main() ");
{
int a=10;
System.out.println("LB1 in main() : "+a);
}
  {
  int a=10;
  System.out.println("LB2 in main() : "+a);
   }
  {
    String a="IND";
    System.out.println("LB3 in main() : "+a);
   }
 }
}
```

**OOPS54.java**

```
class OOPS54{
public static void main(String args[]){
System.out.println("Main() ");
int a=10;
{
int a=50;
System.out.println("LB1 in main() : "+a);
   }
  }
}
```

**OOPS55.java**

```
class OOPS55{
public static void main(String args[]){
System.out.println("Main() ");
   {
   int a=10;
System.out.println("LB in main() : "+a);
   }
int a=20;
System.out.println("Main(): "+a);
  }
}
```

**Write your note here....**

**Program output /error and details:-**

| Prog No. | Output | Error and Details |
|---|---|---|
| OOPS48 | SB: 0<br>LB in inside SB: 0<br>main: 0 | Here, 1st load the class then Static block is execute then statement of main method will be execute. |
| OOPS49 | SB<br>LB in inside SB<br>IB<br>LB in inside IB<br>main | Here, 1st load the class then Static block is execute then Instance block will execute due to object creation then statement of main method will execute. |
| OOPS50 | Main()<br>LB in Main () | We can also define Local block inside the Main() |
| OOPS51 | Main()<br>LB1 in Main ()<br>LB2 in Main () | We can also define Local block inside the Main() |
| OOPS52 | Compilation error | [Error: cannot find symbol a]. Because, variable is use within the scope and also we can't define more than one variable with same name. |
| OOPS53 | Main()<br>LB1 in main() : 10<br>LB2 in main() : 10<br>LB3 in main() : IND | We can define more than one variable with same name but in different scope. |
| OOPS54 | Compilation error | [Error: Variable "a" is already defined]. |
| OOPS55 | Main()<br>LB in main() : 10<br>Main(): 20 | We can define more than one variable with same name but in different scope. |

**Write your note here.....**